PAINEL BMAD

Arquitetura de Referência

SaaS multi-tenant para desenvolvimento assistido por IA com isolamento por projeto

Decisão arquitetural central

Separar o Control Plane do Execution Plane. Laravel/Next.js controlam utilizadores, tenants, projetos, workflows, políticas e auditoria; o código dos clientes é executado apenas em runners isolados e descartáveis, com um único workspace por execução e credenciais temporárias.

Camada	Tecnologia / decisão
Frontend	Next.js
BFF	Next.js server-side; o browser não chama o Laravel diretamente
Backend / Control Plane	PHP + Laravel
Base de dados	MySQL
Fila / coordenação	Redis + Laravel Queue/Horizon (sem expor Redis aos containers de execução)
Infraestrutura	Hetzner Cloud + Private Network + Object Storage S3 compatível
IA	BYO-AI por tenant: OpenAI/Codex, Anthropic/Claude e fornecedor adicional de baixo custo via adapters
Execução	Runner Manager + containers rootless/isolados; workspace exclusivo por projeto/execução
Fonte de verdade do código	Git repository por projeto; runner faz checkout/branch/commit sob credencial temporária

Objetivo: começar simples e economicamente eficiente, sem bloquear crescimento horizontal, isolamento reforçado ou planos empresariais dedicados.

1. Resumo executivo

A plataforma será um SaaS multi-tenant que orquestra o ciclo BMAD, mas não deve executar agentes ou código de clientes dentro do mesmo processo que serve o painel. A arquitetura proposta divide o sistema em dois domínios: Control Plane e Execution Plane.

- Control Plane: identidade, tenants, projetos, billing interno, workflows, estados, artefactos, políticas, credenciais cifradas, auditoria, métricas e orquestração.
- Execution Plane: execução efetiva de Codex/Claude/outros agentes, comandos BMAD, testes, code review e security review em ambientes isolados.
- BFF: o browser conversa apenas com o Next.js. O BFF faz a mediação de sessão, autorização de interface, composição de respostas e chamadas server-to-server para o Laravel.
- Fonte de verdade do código: um repositório Git por projeto. O runner não depende de um volume partilhado permanente entre tenants.
- BYO-AI: cada tenant configura as suas próprias credenciais/provider profiles. Essas credenciais nunca ficam expostas ao browser e só chegam ao runner durante a execução que necessita delas.
- Escalabilidade: aplicação, filas e runners escalam horizontalmente; MySQL, Redis e storage ficam fora do ciclo de vida dos runners.

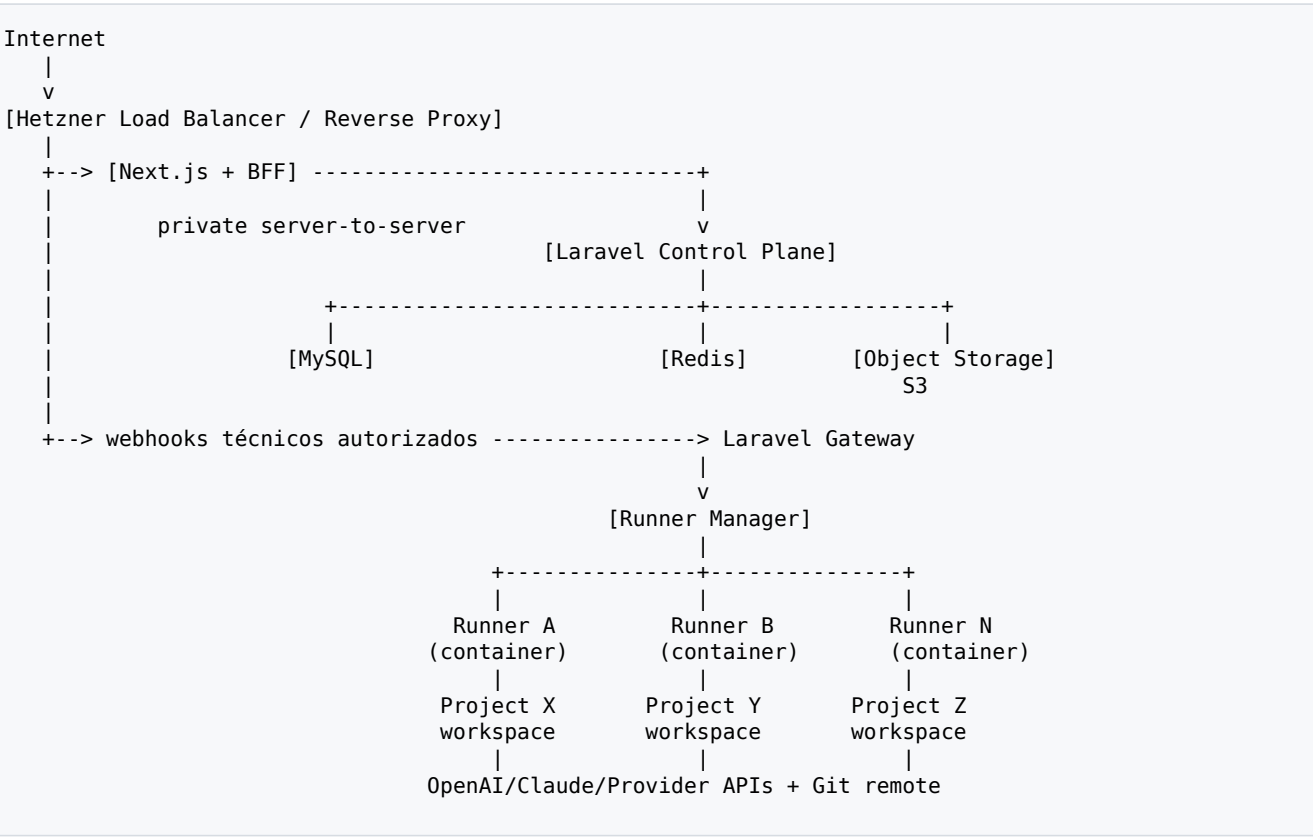
Princípio de segurança
Isolamento lógico no banco de dados não é suficiente para execução de código. O Execution Plane deve considerar o código e os comandos gerados como potencialmente não confiáveis.

2. Objetivos e princípios

Princípio	Aplicação prática
Isolation by default	Nenhum runner recebe paths, tokens ou mounts de outro tenant/projeto.
Least privilege	Cada execução recebe apenas as permissões mínimas e temporárias necessárias.
Stateless compute	Next/Laravel podem ser replicados; runners são descartáveis.
Git as source of truth	Código persistente vive no repositório; workspace é material de trabalho temporário.
Provider-agnostic AI	Workflows dependem de uma interface comum, não de um fornecedor específico.
Auditability	Cada decisão, execução, modelo utilizado, comando, revisão e alteração deve ser rastreável.
Progressive hardening	Começar com Docker/Podman rootless em VMs dedicadas a runners; evoluir para sandboxes mais fortes quando o risco justificar.

Princípio	Aplicação prática
No silent fallback	Nunca enviar código a outro fornecedor de IA sem configuração explícita do tenant.

3. Topologia lógica



O Next.js e o Laravel fazem parte do Control Plane. O Runner Manager pertence ao Execution Plane e deve residir em hosts próprios, sem misturar execução de código de cliente com MySQL, Redis ou servidores da aplicação.

4. Control Plane

4.1 Next.js + BFF

- Único endpoint funcional consumido pelo browser para as operações do produto.
- Mantém sessão em cookie HttpOnly/Secure/SameSite e evita expor tokens internos ao JavaScript do browser.
- Agrega respostas e transforma contratos internos do Laravel em contratos próprios de interface.
- Pode executar streaming de progresso de workflows via SSE/WebSocket, mas sem conceder acesso direto do browser às filas ou runners.
- Downloads/uploads grandes podem usar URLs pré-assinadas; isso é uma exceção controlada ao tráfego via BFF e não expõe a API interna.

Evitar

Transformar o BFF num segundo backend de negócio. Regras de domínio, autorização definitiva e estado de workflow permanecem no Laravel.

4.2 Laravel

- Autoridade de identidade, tenants, memberships, roles e projetos.
- Workflow Engine: estado do ciclo Ideia → Descoberta → Product Brief/Brainstorm → PRD → UX → Arquitetura → Epics → Stories → Desenvolvimento → Validação → Entrega.
- Orquestração das execuções BMAD e respectivos gates.
- AI Provider Registry e políticas de uso por tenant/projeto.
- Gestão de credenciais cifradas, tokens temporários e permissões de execução.
- Audit log, cost/usage ledger, artefactos, relatórios de segurança, ADRs e traceability.
- Integrações Git, deploy e webhooks externos.

4.3 MySQL multi-tenant

Para o Control Plane, a recomendação inicial é shared database / shared schema com tenant_id explícito nas tabelas de negócio. Database-per-tenant aumentaria o isolamento, mas introduziria custo operacional desnecessário para o estágio inicial.

- tenant_id e project_id são derivados do contexto autenticado no servidor; nunca confiados diretamente a valores enviados pelo browser.
- Todas as chaves e índices únicos relevantes devem incluir tenant_id quando o domínio exigir unicidade por tenant.
- Policies/Services/Repositories aplicam escopo obrigatório. Queries sem contexto de tenant devem falhar por padrão.
- Tabelas globais devem ser explicitamente classificadas como globais para evitar exceções implícitas.
- Planos empresariais futuros podem ganhar database dedicada e/ou deployment dedicado sem alterar o modelo conceitual.

Grupo	Exemplos de tabelas
Identidade	users, tenants, tenant_memberships, roles, permissions
Projetos	projects, project_members, project_settings, architecture_decisions
Workflow	workflow_runs, workflow_steps, approvals, gates, retrospectives, guardrails
Execução	executions, execution_events, execution_artifacts, runner_assignments
IA	ai_provider_profiles, ai_provider_credentials, model_policies, usage_ledger
Código / entrega	repositories, branches, deployments, releases, environments
Governança	audit_logs, security_findings, traceability_links

5. Autenticação e fluxo BFF

1. Browser autentica no endpoint do BFF.
2. BFF comunica server-to-server com o Laravel.
3. Laravel valida identidade e memberships e devolve/renova credencial interna de curta duração.
4. BFF guarda a sessão de forma server-side ou num cookie opaco/seguro; tokens de acesso aos serviços internos não ficam disponíveis ao browser.
5. Cada pedido do BFF ao Laravel transporta identidade e escopos verificáveis; Laravel continua a fazer a autorização definitiva.
6. Laravel deriva tenant/project context a partir da sessão e das permissões, não de headers arbitrários do cliente.

Regra
O BFF melhora isolamento de interface e controlo de sessão, mas não substitui autorização no Laravel. Toda operação sensível deve ser novamente autorizada no Control Plane.

6. Camada BYO-AI

O produto deve separar duas coisas que costumam ser misturadas: Provider Credential e Execution Adapter. O primeiro representa como o tenant autentica no fornecedor; o segundo representa como o BMAD executa uma tarefa usando aquele fornecedor.

Componente	Responsabilidade
AIProviderAdapter	Interface comum para capabilities, modelos, streaming, limites, usage e erros.
CredentialProfile	Credencial do tenant, cifrada e associada a um provider.
ModelPolicy	Modelos autorizados por fase, projeto ou custo.
ExecutionAdapter	API direta, Codex CLI, Claude Code/CLI ou outro executor compatível.
Capability Matrix	tool use, filesystem, context size, streaming, structured output, multimodal etc.

Providers iniciais sugeridos como adapters: OpenAI/Codex, Anthropic/Claude e um fornecedor de baixo custo como DeepSeek ou outro compatível. A escolha específica deve ficar desacoplada do domínio.

Atenção sobre “assinatura de IA”
Uma subscrição de produto de consumo não é necessariamente equivalente a acesso de API. O onboarding deve falar em “ligar o seu fornecedor de IA” e suportar API key, OAuth ou mecanismo oficialmente permitido

por cada provider. Não assumir que ChatGPT Plus/Claude Pro fornecem automaticamente créditos de API.

6.1 Proteção de credenciais

- Cifrar credenciais em repouso usando envelope encryption; a master key não deve residir na base de dados.
- Nunca devolver a chave completa ao frontend após o primeiro registo; apenas fingerprint/máscara.
- Desencriptar apenas no momento da execução necessária.
- Injetar a credencial no runner como secret temporário; destruir no final da execução.
- Filtrar secrets de logs, stdout/stderr, relatórios e prompts persistidos.
- Registar provider/model e consumo, mas não conteúdo secreto.

7. Execution Plane e isolamento

Esta é a fronteira de segurança mais importante da plataforma. O Runner Manager recebe jobs autorizados pelo Control Plane e cria um ambiente de execução efémero. O ambiente não deve conhecer a estrutura global do servidor nem ter acesso ao filesystem dos demais projetos.

```
EXECUTION #8f12
  tenant_id: T-104
  project_id: P-882
  workflow: bmad-dev-story

Runner container sees only:
  /workspace                 -> checkout do Project P-882 (RW)
  /opt/bmad/<version>        -> BMAD versionado (RO)
  /tmp                       -> temporário
  execution token            -> escopo Execution #8f12
  AI secret                  -> apenas provider selecionado
  Git credential             -> apenas repo/branch autorizados

Runner must NOT see:
  /tenants/*
  /projects/other-projects
  MySQL credentials
  Redis credentials
  S3 master credentials
  Docker socket
  Laravel APP_KEY
  secrets de outros providers/tenants
```

- Um único workspace de projeto por execução.
- BMAD instalado/versionado fora do workspace e montado read-only.
- Container rootless sempre que viável; sem privileged mode.
- Nunca montar /var/run/docker.sock dentro do container.
- CPU, RAM, disco, PIDs e timeout limitados por execução.
- Concurrency limit por tenant/projeto para evitar abuso e controlar custo de infraestrutura.
- Logs enviados para o Control Plane com execution_id, nunca gravados como fonte de verdade apenas no host do runner.
- Workspace destruído ao final; alterações persistem via Git e artefactos explicitamente publicados.

Evolução do sandbox

Docker/Podman rootless em VMs dedicadas de runners é uma base pragmática. Para projetos de maior risco, pode-se evoluir o isolamento para gVisor, Kata Containers ou microVMs/Firecracker sem mudar o contrato do Runner Manager.

8. Ciclo de vida de uma execução

1. Laravel cria Execution com tenant_id, project_id, workflow, branch/base commit, provider e políticas.
2. Job é colocado na fila interna. O Runner Manager consome a fila; o container de execução não recebe credenciais de Redis.
3. Runner Manager solicita/recebe um execution token de curta duração e credenciais temporárias necessárias.
4. Cria workspace vazio e faz checkout apenas do repositório do projeto.
5. Monta versão BMAD read-only e materializa apenas os artefactos/contextos autorizados.
6. Executa o workflow/agente e transmite eventos/progresso.
7. Executa testes/reviews/security gates definidos para a etapa.
8. Produz diff/commit/artefactos e envia resultados ao Control Plane.
9. Revoga secrets/tokens, limpa o workspace e destrói o container.
10. Laravel decide se a execução avança, exige aprovação, corrige ou bloqueia o workflow.

9. Código-fonte e Git

Git deve ser a fonte de verdade persistente do código. Isso reduz dependência de volumes partilhados, facilita auditoria e permite reconstruir qualquer workspace.

- Um repositório por projeto, preferencialmente privado.
- Branch de trabalho por story/execução ou convenção equivalente.
- Credencial Git temporária e restrita ao repositório necessário.
- Execução produz commit/diff rastreável ao workflow, story e execution_id.
- Merge protegido por gates definidos pelo projeto.
- Provider Git deve ficar atrás de uma interface: GitHub/GitLab/Forgejo ou outro podem ser adicionados sem alterar o workflow.

10. Infraestrutura Hetzner

A Hetzner oferece Cloud Servers, Private Networks, Firewalls, Load Balancers, Volumes, Backups e Object Storage S3-compatible. A arquitetura deve usar esses blocos sem acoplar o domínio à API da Hetzner.

Zona	MVP recomendado	Escala
Edge	Reverse proxy no app node ou LB quando necessário	Hetzner Load Balancer + múltiplos app nodes
App	1 VM: Next.js + BFF + Laravel web	2+ app nodes stateless
Workers Control Plane	Pode coexistir na VM de app	VMs separadas para queue/Horizon

Zona	MVP recomendado	Escala
	inicialmente	
MySQL	VM privada dedicada + backups	Primary + réplica/read replica/manual failover conforme crescimento
Redis	Pode iniciar junto ao app se custo for crítico	VM privada dedicada / HA conforme criticidade
Runner Pool	1 VM separada do app/DB, múltiplos containers isolados	N VMs de runner auto-provisionadas por fila/concurrency
Object Storage	Artefactos, exports, backups, anexos	Mantém-se horizontal e fora dos runners

Rede

MySQL e Redis devem ficar na Private Network e sem exposição pública. Runners precisam de saída para providers de IA, Git e package registries; no MVP podem ter interface pública com firewall bloqueando inbound. Em estágio posterior, centralizar egress num gateway/proxy permite regras e auditoria mais fortes.

Object Storage

Não fornecer credenciais S3 globais ao runner. Na Hetzner, uma access key/secret key pode, por defeito, alcançar os buckets do mesmo projeto. Prefira operações via Control Plane ou URLs pré-assinadas e de curta duração para objetos específicos.

11. Segmentação de rede

```
PUBLIC
  80/443 -> Edge / App
  webhooks externos -> endpoint dedicado e autenticado

PRIVATE NETWORK
  App/BFF <-> Laravel services
  Laravel -> MySQL
  Laravel/Workers -> Redis
  Runner Manager -> Control Plane Runner API

RUNNER EGRESS
  HTTPS -> AI provider endpoints
  HTTPS/SSH -> Git provider (conforme configuração)
  HTTPS -> package registries aprovados
  HTTPS -> Control Plane scoped endpoints

DENY
  Runner -> MySQL
  Runner -> Redis
  Runner -> host Docker socket
```

```
  Runner -> outros runners/workspaces
```

A allowlist total de egress pode dificultar builds no início devido à quantidade de package registries e downloads indiretos. A arquitetura deve permitir começar com outbound controlado e evoluir para um egress proxy com política por projeto/plano.

12. Estratégia de storage

Dado	Fonte de verdade	Observação
Metadados SaaS	MySQL	tenant-scoped
Sessões / fila / cache	Redis	não é fonte persistente final
Código	Git	repo por projeto
Workspace de execução	Disco efémero local do runner	destruído ao final
Artefactos BMAD	Object Storage + metadata MySQL	versionar/hash quando relevante
Uploads do utilizador	Object Storage	acesso por URL pré-assinada
Logs/auditoria	MySQL/stream de logs + storage de retenção	imutabilidade lógica e retention policy
Backups	Object Storage / mecanismo externo	separados do ciclo operacional

13. Orquestração dos workflows BMAD

Cada etapa BMAD deve ser representada por um workflow declarativo no Control Plane. O workflow determina inputs, executor, provider/model permitido, artefactos esperados, reviewers, gates e transições.

```
WorkflowStep
  id
  workflow_id
  step_type          # discovery | prd | ux | architecture | create-story | dev-story | ...
  executor_type      # llm-api | codex-cli | claude-cli | internal-service
  input_contract
  output_contract
  required_artifacts
  guardrails[]
  review_policy
  security_policy
  approval_policy
  retry_policy
  timeout
```

Isso evita codificar o comportamento de cada agente diretamente na interface e permite trocar provider, modelo ou executor sem alterar o fluxo de negócio.

14. Rastreabilidade e auditoria

- Discovery / PRD requirement → UX flow → ADR → Epic → Story → Execution → Commit → Test → Security Finding → Release.
- Cada execution guarda provider, model, versão BMAD, versão de guardrails e hash/base commit.
- Alterações manuais do utilizador e aprovações devem ficar registadas com identidade e timestamp.
- Retrospectivas podem gerar guardrails versionados; execuções futuras registam qual versão utilizaram.
- Relatórios de segurança devem preservar finding, severidade, correção e estado.

15. Estratégia de crescimento

15.1 Fase 1 - MVP

- 1 App VM: Next.js + BFF + Laravel web.
- 1 Data VM privada: MySQL; Redis pode começar na App VM se custo for prioritário, mas com plano de separação.
- 1 Runner VM separada, com containers rootless/isolados e limites de recursos.
- Object Storage para artefactos e backups.
- Git externo ou serviço Git próprio isolado.
- Backups automáticos + backup lógico de MySQL fora da própria VM.

15.2 Fase 2 - crescimento

- Hetzner Load Balancer.
- 2+ App VMs stateless.
- Workers Laravel separados.
- Redis dedicado.
- Runner Pool com múltiplas VMs e scheduler por capacidade.
- Métricas, tracing e alertas centralizados.
- Egress proxy para runners.

15.3 Fase 3 - enterprise / isolamento reforçado

- Runner pool dedicado por tenant ou execução em microVM.
- Database dedicada opcional.
- Storage/keys dedicados quando exigido contratualmente.
- Região/infraestrutura dedicada conforme compliance.
- Políticas de egress estritas e integração com SIEM.

16. Decisões que devem ser evitadas

Evitar	Motivo
Executar Codex/Claude no mesmo host do MySQL	Aumenta severamente o impacto de escape ou comando indevido.
Uma pasta /clients com todos os clientes montada nos runners	Isolamento por convenção de path é frágil.
Dar credencial Redis/MySQL ao container do agente	Um prompt/code execution comprometido passa a ter

Evitar	Motivo
	acesso ao control plane.
Usar apenas tenant_id enviado pelo frontend	Permite IDOR/cross-tenant se o backend não derivar e autorizar contexto.
Gravar API keys em texto ou logs	Expõe credenciais de terceiros e potencial custo financeiro.
Acoplar workflow a Claude/Codex diretamente	Dificulta BYO-AI e mudança de provider.
Persistir workspace como única fonte de verdade	Complica recuperação, auditoria e escalabilidade horizontal.
Introduzir Kubernetes no MVP sem necessidade	Custo operacional superior ao benefício inicial; runners podem escalar por VMs + containers.

17. ADRs iniciais recomendados

ADR	Decisão
ADR-001	Control Plane e Execution Plane separados.
ADR-002	Next.js atua como BFF; browser não consome Laravel diretamente.
ADR-003	Laravel é a autoridade de domínio, workflows e autorização.
ADR-004	MySQL shared-schema multi-tenant com tenant_id obrigatório no Control Plane.
ADR-005	Código persistente em Git; workspace de runner é efémero.
ADR-006	BMAD versionado e read-only dentro da execução.
ADR-007	Runner descartável por execução com secrets temporários.
ADR-008	AI Provider abstraction + BYO-AI por tenant.
ADR-009	Object Storage usado para artefactos/backups; runners não recebem credenciais globais.
ADR-010	Deploy target definido desde a arquitetura do projeto e tratado por adapters.

18. Componentes sugeridos no código

Estrutura conceitual do backend Laravel:

```
app/
  Domain/
    Identity/
    Tenancy/
    Projects/
    Workflows/
    AI/
    Executions/
    Repositories/
    Architecture/
    Security/
    Deployments/
    Audit/
  Application/
  Infrastructure/
    AI/Providers/
    Git/Providers/
    Storage/
    Runners/
    Hetzner/
  Http/
```

Estrutura conceitual do Next.js:

```
src/
  app/
  features/
  bff/
    auth/
    projects/
    workflows/
    executions/
  server/
    api-client/
    session/
    streaming/
  components/
```

O diretório BFF deve conter composição e adaptação de interface, não regras de domínio que pertençam ao Laravel.

19. Checklist de segurança arquitetural

- MFA disponível para contas administrativas e, idealmente, tenants.
- RBAC/ABAC com tenant/project scopes.
- CSRF, secure cookies, rate limiting e proteção de sessões no BFF.
- Tokens internos de curta duração e rotacionáveis.
- Secrets cifrados e mascarados.
- Runner sem acesso direto a DB/cache/global storage credentials.
- No Docker socket inside workload containers.
- Resource limits e timeouts obrigatórios.
- Antivirus/malware scanning para uploads quando aplicável.
- Dependency/security scanning dentro dos gates BMAD.
- Audit trail append-oriented para alterações de segurança e permissões.
- Backups testados com restore periódico.
- Política de retenção/deleção por tenant e projeto.

20. Deploy dos projetos dos clientes

O deploy do SaaS Painel BMAD e o deploy dos projetos criados pelos clientes são problemas diferentes. O segundo deve ser abstraído por Deployment Adapters.

Interface	Exemplos
DeploymentProvider	Hetzner, VPS Docker, Kubernetes, provider externo futuro
Environment	dev, staging, production
SecretSet	secrets por ambiente/projeto
DeploymentPlan	build, migrations, rollout, health check, rollback
DeploymentRun	execução auditável com commit/release associado

Regra
O runner de desenvolvimento não deve possuir automaticamente credenciais permanentes de produção. Deploy é uma capability separada, com aprovação e secret scope próprios.

21. Recomendações de implementação

1. Implementar primeiro a fronteira Tenant → Project → Execution antes de integrar vários providers de IA.
2. Construir o Runner Manager como serviço próprio com contrato estável; não espalhar comandos Docker pelo Laravel.
3. Definir AIProviderAdapter e ExecutionAdapter desde o início, mesmo que apenas Codex esteja ativo no primeiro release.
4. Usar Git por projeto antes de permitir desenvolvimento real; não usar filesystem partilhado como sistema de versionamento.
5. Criar execution token com escopo explícito para cada job e expiração curta.
6. Instrumentar audit log e execution events desde o MVP; adicionar depois é muito mais difícil.
7. Separar deployment credentials das AI/Git credentials.
8. Tratar Security Review como gate do workflow, não como simples relatório opcional.
9. Evitar microserviços no Control Plane no início. Laravel modular + workers + Runner Manager já cria separação suficiente para crescer.
10. Fazer testes adversariais de cross-tenant: IDs manipulados, path traversal, runner escape, secret leakage e acesso a repo errado.

22. Critérios mínimos para considerar a arquitetura pronta para o MVP

- Um utilizador do Tenant A não consegue obter Project/Execution/Artifact do Tenant B por manipulação de IDs.
- Um runner do Project A não possui mount, token ou path que permita ler Project B.
- Eliminar o container elimina o workspace local sem perder código já persistido no Git.
- Nenhuma API key de IA aparece no browser, audit log ou stdout persistido.
- Trocar Codex por Claude numa etapa não exige alterar a regra de negócio do workflow.
- Control Plane continua operacional se um runner falhar ou for destruído.
- Novas Runner VMs podem ser adicionadas sem alterar Next.js/Laravel.
- MySQL/Redis não aceitam conexões públicas da internet.
- Cada execução pode ser ligada a tenant, projeto, story, provider/model, commit, reviews e resultados.
- Restore de base de dados e artefactos possui procedimento testável.

23. Referências técnicas verificadas

As decisões acima usam capacidades atuais da Hetzner verificadas em setembro de 2026. A documentação da Hetzner confirma suporte a Cloud Servers, Private Networks, Firewalls, Volumes, Backups, Load Balancers e Object Storage compatível com S3. Private Networks permitem comunicação por IP privado; Load Balancers podem encaminhar para targets privados; Object Storage usa credenciais S3 e é adequado para artefactos/backups, não como filesystem de workspace.

Hetzner Cloud Servers - Overview: https://docs.hetzner.com/cloud/servers/overview/

Hetzner Networks - Overview: https://docs.hetzner.com/networking/networks/overview/

Hetzner Load Balancers - Overview: https://docs.hetzner.com/networking/load-balancers/overview/

Hetzner Object Storage - Overview: https://docs.hetzner.com/storage/object-storage/overview/

Hetzner Object Storage - Buckets & Objects FAQ: https://docs.hetzner.com/storage/object-storage/faq/buckets-objects/

24. Arquitetura recomendada em uma frase

Síntese

Um Control Plane modular em Laravel + Next.js/BFF, multi-tenant em MySQL, coordena workflows e políticas; cada tarefa de desenvolvimento é executada num runner isolado e descartável com um único repositório/workspace, BMAD read-only e credenciais temporárias do provider de IA escolhido pelo cliente, sobre infraestrutura Hetzner preparada para escalar horizontalmente.